KNN & SVM 对照实验与适用性判断

王羿珧 （学号 2026211232）

1 算法实现

本报告选择了 KNN 与线性软间隔 SVM。前者假设邻居属于同类别，后者假设边界是线性超平面且以最大间隔分隔两类。通过合成数据与真实数据，本报告尝试探究二者的差异并给出适用性判断。两个算法的核心训练与预测只依赖 NumPy，绘图使用 Matplotlib。

1.1 KNN 的实现

距离矩阵用展开式 $\|x-t\|^2 = \|x\|^2 + \|t\|^2 - 2x\cdot t$ 通过 Numpy 广播机制算出，避免使用计算缓慢的 Python 循环。每个查询行的前 k 近邻用 `argpartition` 以 $O(n)$ 取得，不做全排序。投票方式有两种，代码里分别是 `uniform` 与 `distance`，对应均匀投票与按距离加权投票。加权投票的权重取 $1/d$，分母加下界 10^{-12}，避免 d=0 时分母为 0。

1.2 SVM 的实现

求解

$$\min_{w,b}\ \frac{1}{2}\|w\|^2 + \frac{C}{n}\sum_{i=1}^{n}\max\bigl(0,\ 1 - y_i(w\cdot x_i + b)\bigr), \tag{1}$$

采用全批量梯度下降，配合回溯线搜索、梯度范数裁剪与相对容差 10^{-6} 的早停，并用 `fit` 记录完整损失序列。其中线搜索每次将步长减半，至多尝试 60 次。损失函数中，hinge 项按样本数取均值，等效惩罚系数为 C/n。

1.3 正确性检查

实现正确性通过与 sklearn 的平行对照检验，脚本为 `experiments/verify_sklearn.py`，实验结果被保存在 `results/csv/verify_sklearn.csv`，逐行明细见附录表 A1 与表 A2。

2 合成数据实验

四组实验共用了一套设置，在此先进行统一说明。数据生成后按 3:1:1 切分为训练、验证与测试集，种子为 42。实验超参数仅由验证集准确率选出，测试集只评估一次。下文表格中的数值都是测试集准确率，其它指标例如 F1 只在正文里单独说明。KNN 网格为 $k \in \{1,3,5,7,9,11,15,21\}$，实验 4 的 k 网格加密到 51。SVM 网格为 $C \in \{10^{-1},\dots,10^{6}\}$。对于每组实验生成的图片，左子图是测试集按真实标签着色并叠加真实边界，中间与右子图是 KNN 与 SVM 的预测，错误分类的点标记为 $\times$，背景为逐点预测着色的决策区域。

2.1 实验 1　线性可分数据

数据由 $x_1 = 2y + \varepsilon$，$\varepsilon \sim N(0,\sigma^2)$，$x_2 \sim N(0,\sigma^2)$ 生成，真实标签 $y \in \{+1,-1\}$，$n = 1200$，真实边界是 $x_1 = 0$。σ 取 0.5、1.0、1.5 控制重叠程度。由于数据是线性可分或近似现行可分的，实验预期两个算法有类似的、较好的表现，其中 SVM 的全局直线边界与真实

表 **1** 实验 1、2 的测试集准确率。实验 2 的单位圆为 $r \le 1$，外环为 $1.4 \le r \le 2.4$

实验	设置	KNN	SVM
实验 1	$\sigma = 0.5$	1.000（k=1）	1.000（C=0.1）
	$\sigma = 1.0$	**0.975**（k=11）	0.971（C=0.1）
	$\sigma = 1.5$	**0.929**（k=9）	0.917（C=1.0）
实验 2	单位圆与外环	**1.000**（k=1）	0.629（C=100）

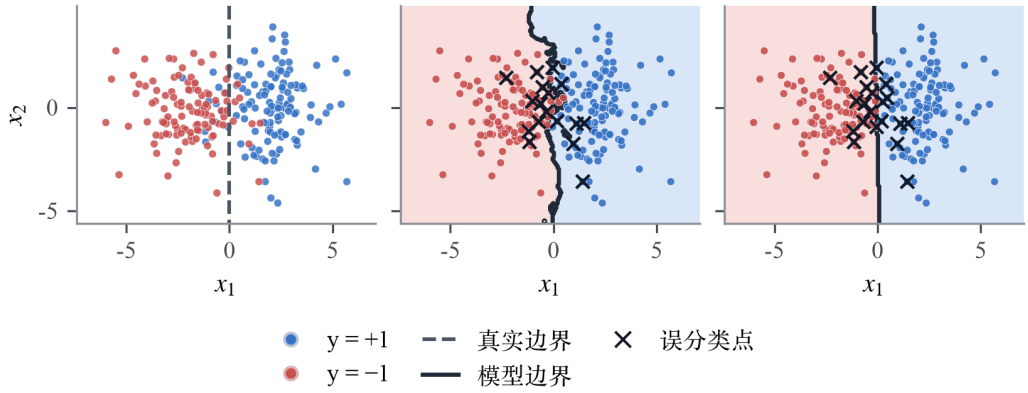


图 **1** 实验 1 在 σ=1.5 下的决策边界，其模型超参由验证集选出。左子图为真实标签与边界 $x_1 = 0$。中间 KNN 的边界是多数投票切换处，锯齿集中在重叠带内，测试准确率 0.929。右子图 SVM 的边界是超平面 $w \cdot x + b = 0$，测试准确率 0.917。

边界形状一致。当重叠比例较大时，预期 SVM 会略不占优，因为此时边界不再完全线性可分。

实验结果见表 1 上半部分与图 1。在 $\sigma = 0.5$ 时两者准确率都是 1.00，$\sigma = 1.0$ 时 KNN 为 0.975，SVM 为 0.971，而 $\sigma = 1.5$ 时 KNN 为 0.929，SVM 为 0.917。图 1 中 KNN 的边界呈竖直走向的锯齿，锯齿只出现在两类重叠的竖直带内，而 SVM 的边界则是一条穿过重叠带中央的直线。

2.2 实验 2 非线性边界

普通 SVM 无法处理好非线性边界，下面通过二维合成数据的实验验证。具体来说，单位圆 $r \le 1$ 内部的点为 $+1$ 类，外环 $1.4 \le r \le 2.4$ 中的点为 -1 类，数据沿着半径均匀采样 $n = 1200$ 个样本。两类之间留有 $[1.0, 1.4]$ 的空隙，真实边界为 $r = 1.2$。实验预期 KNN 的准确率接近 1.00，而线性 SVM 无论怎么放直线都切不开圆盘与圆环，准确率会比较差。

结果见图 2 与表 1 末行。KNN 的测试准确率 1.000，选出最优超参为 k=1。由于空隙宽达 0.4，最近邻几乎必然是同类，因此 k=1 就足够。与之相对，SVM 的准确率只有 0.629，查准率 0.568，查全率 0.885。最优解把约七成测试点划入 $+1$ 一侧，尽管抓住了大部分单位圆内的点，但也把大量外环点误判了进来。观察图 2 可见，KNN 的决策区域能够准确划分数据，而 SVM 只能用一条直线切出半个平面，包不住环形结构。

2.3 实验 3 高维数据

KNN 无法处理好高维度数据，特别是当维度中有多个噪声维度时。这是因为二范数会考虑所有维度，而噪声维度冲淡了真正能够区分类别的维度，让计算出的欧几里得距离不再能作为度量类别的标准。本实验构造仅维度 1 携带信号，$x_1 = 1.5y + \varepsilon$，$\varepsilon \sim N(0, 0.5^2)$，其余 $d - 1$ 维均为 $N(0, 5^2)$ 噪声（专门设置噪声比信号有更大的方差，从而能够更早观察到 KNN 无法应对高维数据），$n = 1000$。维度 d 从 3 取到 1000 共七档。实验预期噪声维度会

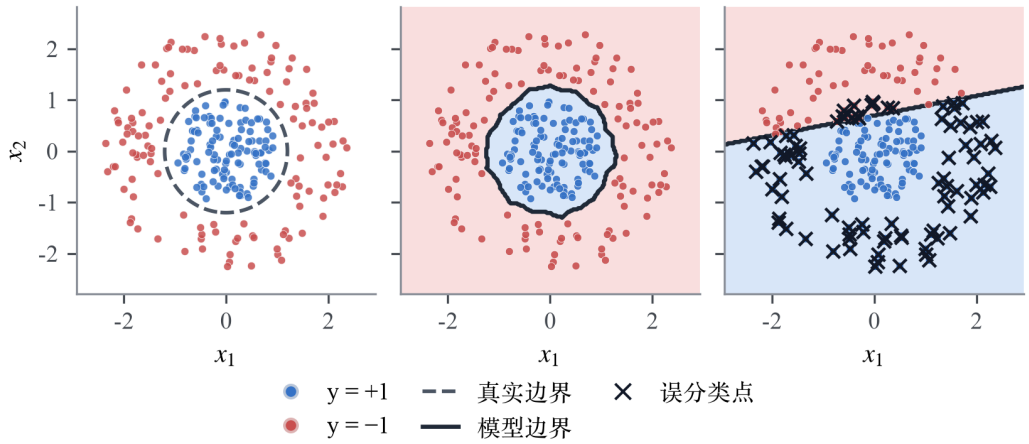


图 **2** 实验 2 的决策边界。左子图为真实标签，中间为 KNN，右子图是 SVM。KNN 选出 k=1，决策区域弯成环带，准确率 1.000。线性 SVM 只能切出半个平面，准确率 0.629。

表 **2** 实验 3 测试集准确率

d	3	10	50	100	200	500	1000
KNN 准确率	0.985	0.685	0.595	0.555	0.500	0.505	0.525
KNN 选出 k	3	21	1	7	21	21	21
SVM 准确率	**0.995**	**1.000**	**0.995**	**0.990**	**0.840**	**0.530**	**0.530**
SVM 选出 C	1	1	10	10	10	10^3	0.1

稀释欧几里得距离的对比度，KNN 随 d 退化。而 SVM 的梯度在噪声维度上期望为零，依然能够在高维度利用有效信号做分类。

结果见表 2 与图 3。其中，KNN 的测试准确率从 d=3 的 0.985 掉到 d=10 的 0.685，d=200 起与随机猜测无异，停在 0.5 附近。可以估算，两类别中心在维度 1 上相距 3，信号对距离平方的贡献约为 9。但每个噪声维度的贡献却能达到 25，d 维合计约 $25d$。最近邻与第 50 近邻的距离对比度约为 $9/(9+25d)$，在 d=50 时就只剩 0.7% 了。与之相对，SVM 的测试准确率在 $d \le 50$ 时不低于 0.995，d=100 时为 0.99，d=200 时为 0.84，到 d=500 与 d=1000 时才明显受损，准确率分别为 0.53 与 0.53。此时 600 个训练样本撑不起 d 维权重。

图 3 给出了 d=200 的三维视角，其中 SVM 的超平面切片几乎与真实边界 $x_1 = 0$ 重合，但 KNN 的预测已经对不上正确标签，出现大面积的错误。

2.4 实验 4 系统性标签噪声

线性软间隔 SVM 会受到少量异常点的影响。为此，本实验专门构造一些系统性的标签翻转，来观察 SVM 和 KNN 对这些噪声的鲁棒性。首先，纯净数据的构造规则为 $x_1 + 0.5x_2 \ge 0$，$n = 2500$。接下来，实验注入 η 比例的标签翻转点，它们的类别被翻转，η 从 0 取到 0.12 共五档。翻转点成对分布在真实边界的另一侧，深度 s 服从均匀分布 $U(1.0, 1.6)$，沿边界的投影 t 服从 $N(\pm 3, 1)$。与随机翻转不同，这些点对边界产生方向一致的推力。实验预期 SVM 的超平面被扭转，准确率随 η 单调下降，且 C 越大扭转幅度越大，而 KNN 只在错标簇附近局部受损。

实验结果见表 3 与图 4。首先，KNN 的准确率几乎没有下降，从 η=0 的 0.980 到 η=0.12 的 0.964，各档 η 下验证集选出的最优 k 恒为 5。对于 KNN 而言，错标只污染点周围的邻域，离翻转点较远的查询点周围仍是同类点。与之相对，SVM 的准确率则从 0.994 跌到 0.842，在图 4 右子图中，可以观察到超平面被略微顺时针扭转了一定的角度，导致边界附近的点被错误分类。

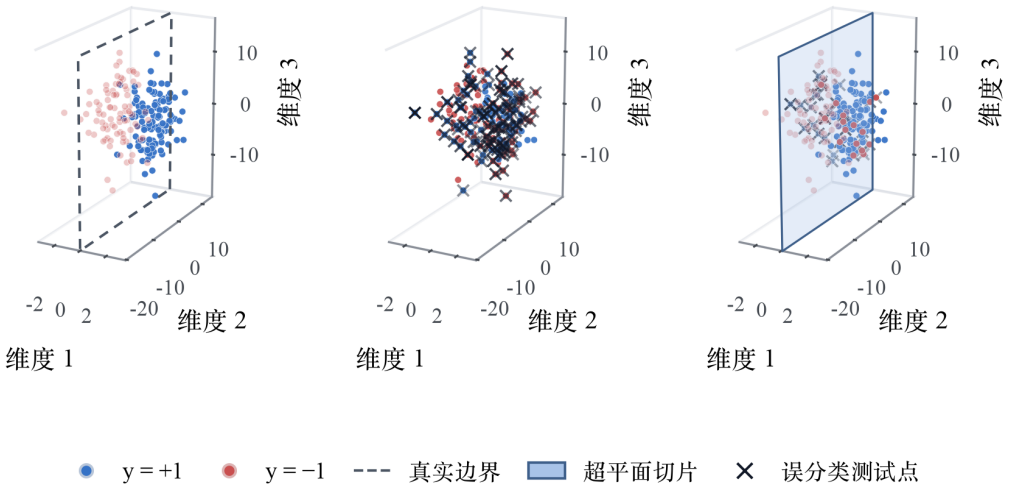


图 **3** 实验 3 在 d=200 时的三维视角，从左到右为真实标签、KNN 与 SVM。轴为维度 1 至 3 且维度 1 是唯一有效维度。SVM 子图中半透明平面为超平面在其余维度取测试集均值处的切片，虚线框为真实边界 $x_1 = 0$，两者几乎重合，测试集准确率达到 0.840。KNN 的预测着色已对不上真实标签，准确率只有 0.500。

表 **3** 实验 4 测试集准确率与超参数 C 的选择

η	0	0.02	0.05	0.08	0.12
KNN 准确率	0.980	**0.976**	**0.970**	**0.966**	**0.964**
SVM 准确率	**0.994**	0.966	0.928	0.882	0.842
SVM 选出 C	10^4	10^3	10	10	10^3

不过，实验结果存在两处与预期不符。其一，C 越大扭转幅度越大的现象并未出现，$\eta \geq 0.05$ 时准确率随 C 变化的曲线在 $C \geq 1$ 段比较平坦，η=0.12 时 C=1 准确率 0.806、C=10^6 准确率 0.844，这可能是因为翻转点紧贴间隔带，软硬间隔均无法绕开这些点，C 的作用被 η 所掩盖。其二，C=0.1 的正则强度过强，模型直接塌成多数类预测，在纯净数据上准确率也只有 0.502。总的来说，面对这些系统性污染，调整正则项系数 C 不能完全解决问题，清理训练数据或更换局部模型是更加本质的办法。

3 算法机制展示

在前面的部分中，报告已经展示了决策边界并对形状进行了解释。本部分将针对 KNN 和 SVM 再各深入一个算法细节，进行机制的展示。具体来说，KNN 关注邻居选择的机制，SVM 关注间隔与正则化系数 C 的作用。

3.1 KNN 的邻居选择

图 5 在一个刻意构造的邻域里解剖单点预测。查询点为五角星，真实标签 $+1$。查询点周围的邻居按距离排序，最近 2 个为 $+1$，第 3 至 5 个为 -1，第 6 至 10 个为 $+1$，第 11 个为 -1。当选择 k=5 时，均匀投票 2 比 3，模型预测类别为 -1，分类错误。但采用距离加权后，最近 2 个 $+1$ 的权重和占优，模型预测类别为 $+1$，分类正确。当选择 k=11 时，投票比例为 7 比 4，两种投票方式均预测正确。总的来说，加权投票相当于把有效邻域向近处收缩，让距离更近的邻居更重要，从而与均匀投票给出不同答案。

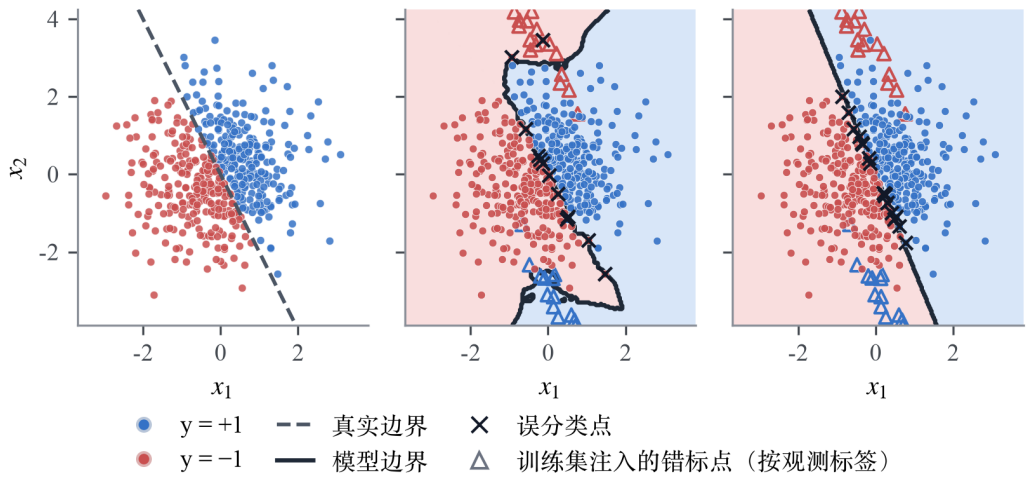


图 **4** 实验 4 在 η=0.02 下的决策边界。左子图为真实标签，中间为 KNN，右子图是 SVM。空心三角为注入训练集的翻转点，按观测标签着色，KNN 的边界只在错标点附近出现变化，SVM 的超平面则整体向错标方向偏转，虚线为真实边界 $x_1 + 0.5x_2 = 0$。

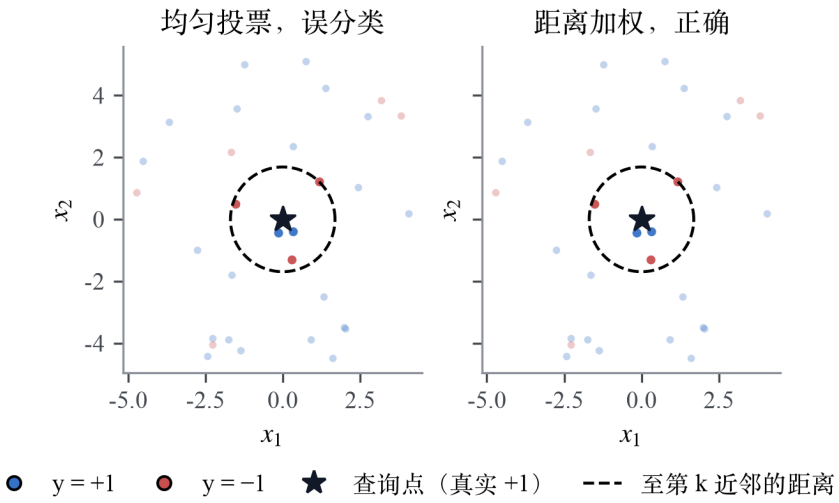


图 **5** KNN 实例级邻居选择。黑色虚线圆表示直至第 k 近邻的邻域半径。左子图采用均匀投票，预测为红色，分类错误。右子图采用距离加权，预测为蓝色，分类正确。

图 6 把 KNN 超参数 k 放到完整决策区域上看，训练标签含 8% 的随机翻转。k=1 时训练集准确率 1.000、验证集 0.822、测试集 0.794，决策区域里有较多追逐翻转点的孤岛。而 k=11 时，准确率在验证集上达到 0.894、测试集 0.883，k=21 时进一步到 0.906 与 0.889。对于随机噪声，用更大的邻域可以将这些翻转平均掉，但代价是局部结构被抹平，同时计算开销更大。

3.2 SVM 的间隔与 C

图 7 是 SVM 间隔的机制展示图，数据设定 n=160、C=100，间隔宽 $2/\|w\| = 1.22$，满足 $y\,(w \cdot x + b) \le 1$ 的支持向量有 9 个，占 6%。其余点离边界足够远，对 w 的梯度为零、对解没有贡献。支持向量的稀疏性意味着模型可以用少数点解释，这体现了 SVM 的可解释性。

图 8 让 SVM 超参数 C 从 10^{-2} 扫到 10^{6}，数据设定 n=1500、训练集包含 900 个样本。可以观察到，间隔宽从 110 整体降到 1.73 附近，支持向量数从 900 降到 166 后饱和，占

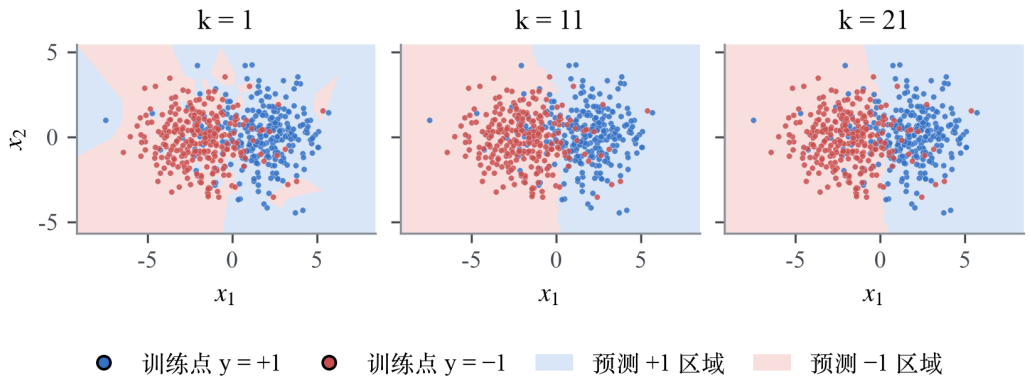


图 **6** KNN 按 k 扫描。从左到右依次为 k=1、11、21，训练标签含 8% 随机翻转。k=1 时模型会追逐翻转点，从而出现一个个孤岛。从评价指标来看，k=21 最好，k=11 略低。

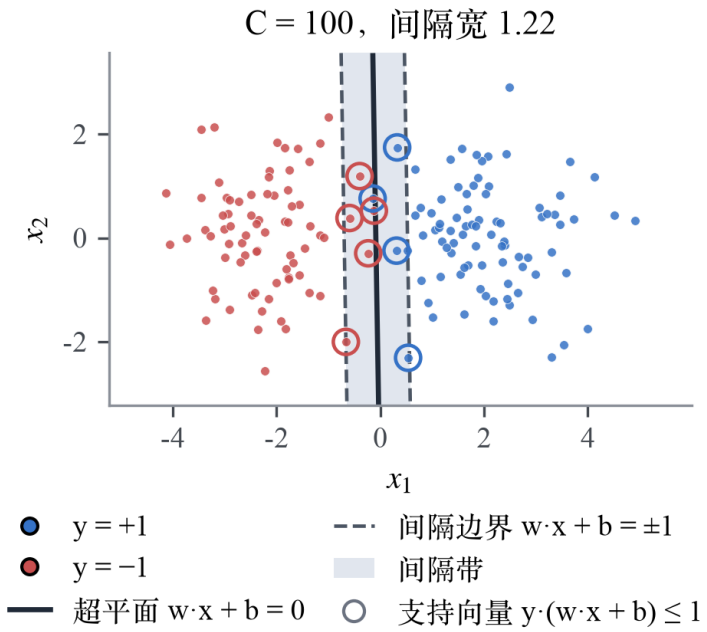


图 **7** SVM 间隔。实线为超平面，虚线为间隔边界 $w \cdot x + b = \pm 1$，阴影为间隔带，空心圈为支持向量，共 9 个，间隔宽 $2/\|w\| = 1.22$。

18%。C=0.1 时几乎所有点都落在间隔带内。C=10^4 时只剩不到两成的点顶住边界。与此同时，训练集、验证集、测试集准确率变化不大，验证集在 0.883 至 0.893 之间，测试集在 0.907 至 0.937 之间。由此可见，数据近似线性可分时，C 对分类结果影响不大，与合成数据实验 4 的结果一致。

4 真实数据集与超参数搜索

4.1 实验设置

两个数据集、两个模型采用完全相同的实验流程。webspam 的 unigram 版有 30,000 条样本，正类占 60.7%，本报告使用全量数据。而 HEPMASS 有 500,000 条样本，其中正类占 50.1%，本报告按类别分层、无放回抽 100,000 条，随机种子为 42，类别比例与全量一致。抽样是出于计算量考虑。数据的切分比例为 3:1:1、随机种子为 42，对应 webspam 的

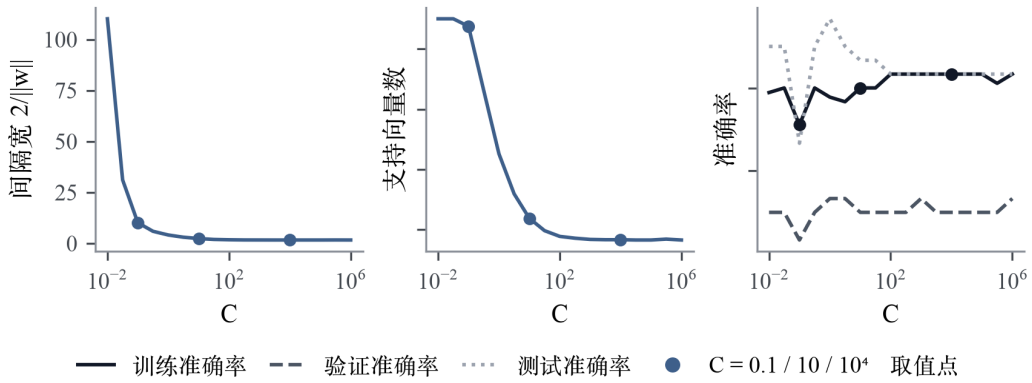


图 **8** SVM 按 C 扫描。从左到右为间隔宽、支持向量数与准确率随 C 的变化，圆点标出 $C = 0.1$、10、10^4 三个取值。只针对本次展示的数据来看，准确率对 C 并不敏感。

18000/6000/6000 与 HEPMASS 的 60000/20000/20000。标准化只在训练集上估计均值与标准差，训练集零方差列丢弃后 webspam 保留 127 维，同一变换作用于验证与测试集。Grid Search 方面，KNN 的 k 在 webspam 上取 1 至 51，在 HEPMASS 上取 1 至 101，计算方法包括纯投票与按距离加权的投票。SVM 的 C 从 10^{-2} 取到 10^{6}。超参数选择只看验证集的准确率，并列时取 F1 较高者。测试集则只在最终评估时出现一次。

评估指标采用准确率与 F1 这两个分类任务常见的经典指标，两个数据集的正类分别为垃圾页与信号事件。

4.2 实验结果

表 4 汇总了最终测试结果，图 9 与图 10 则给出搜索过程。

可以看出，webspam 上 KNN 更强。它的验证准确率整体随 k 下降，最优值取在 k=3、按距离加权的超参数组合上，为 0.966，均匀加权除 k=1 处持平外，其余各档都更低。SVM 的验证准确率随 C 上升并收敛，$C \geq 10^3$ 后稳定在 0.934 附近，最高为 C=10^6 处的 0.935。C=0.1 是异常点，验证准确率只有 0.649。最终 KNN 以 k=3 距离加权的超参数组合取得准确率 0.9645、F1 0.9711，查全率高达 0.985，垃圾页几乎不漏。SVM 以 C=10^6 取得准确率 0.9307、F1 0.9429。

HEPMASS 上两个模型几乎打平。KNN 的验证准确率随 k 从 0.860 一路升至 k=51 的 0.909，之后缓慢回落，k=101 时仍有 0.907，均匀与距离两条曲线几乎重合。SVM 的验证准确率自 C=100 起慢慢收敛为 0.911。测试集上 SVM 以准确率 0.9071、F1 0.9084 略胜 KNN 的 0.9036、0.9052。6 万训练样本下最优 k 停在 51。

表 4 的耗时列展示了算法的开销。HEPMASS 的 KNN 预测耗时 13.1 秒，而 SVM 在最优参数下训练不到 1 秒、预测在万分之一秒量级。KNN 的优势建立在不做任何训练上，时间成本则转移到预测阶段。

表 **4** 真实数据的测试集准确率、F1 与耗时

数据集	模型	超参	准确率	F1	耗时（秒）
webspam	KNN	k=3，距离加权	**0.9645**	**0.9711**	训练 0.00，预测 0.68
webspam	SVM	C=10^6	0.9307	0.9429	训练 0.70，预测 0.0001
HEPMASS	KNN	k=51，距离加权	0.9036	0.9052	训练 0.00，预测 13.12
HEPMASS	SVM	C=100	**0.9071**	**0.9084**	训练 0.05，预测 0.0001

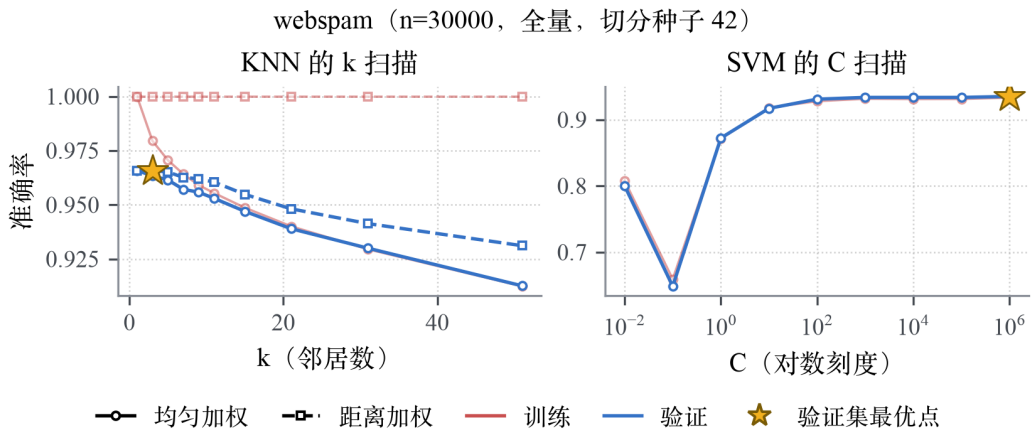


图 **9** webspam 上的超参搜索曲线，训练集与验证集准确率随超参数变化，星标为验证集选出的最优点。左子图 KNN 中实线为均匀加权、虚线为按距离加权，右子图为 SVM。

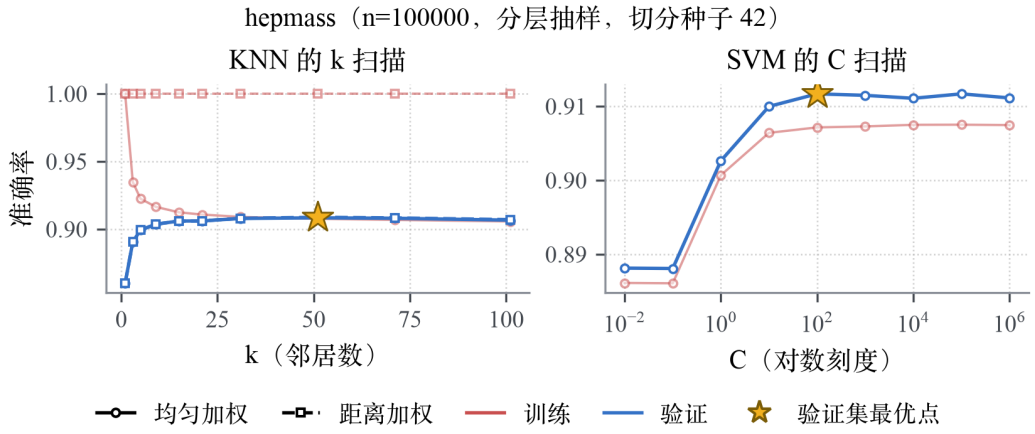


图 **10** HEPMASS 上的超参搜索曲线，含义同图 9。

5 适用性判断

首先，选择方法时需要预估分类问题中类别间边界的形状。低维、边界非线性、样本密度高时应当选择 KNN，证据是合成数据实验 2。与之相对，边界近似线性、或需要全局一致的决策规则时选 SVM。

其次，数据的噪声维度变大可能会让距离不再适合作为分类标准，从而不适合使用 KNN。合成数据实验 3 表明，信号维度少而噪声维度多时，距离对比度随 d 塌缩，KNN 分类错误率大幅度提高。但高维不必然导致 KNN 失效，关键是维度是否有意义。webspam 有 127 维，KNN 的准确率仍领先 SVM 约 3.6%，因为标准化后各维都携带有效的判别信息。总结来说，信息密集在少数维、其余是噪声时选 SVM，信息分散在多数维且距离有区分度时 KNN 仍可用。

第三，两种方法针对标签噪声有不同的应对方式。如果训练标签是被随机翻转的，每个样本都有相同概率被改错，KNN 只要把 k 调大一些即可。实验 5（算法机制展示）在训练集里注入了 8% 的随机翻转，k 从 1 加到 11 之后，测试准确率从 0.79 升到 0.88，更大的邻域把零星的反向标签平均掉了。另一种是系统性污染，错标样本成对地贴在真实边界的另一侧，方向一致，会合力推动决策面。合成数据实验 4 中这种污染让 SVM 的准确率从 0.994 掉到 0.842，扫遍所有超参数 C 也无法挽救，因为错标数据紧贴间隔带。但 KNN 则几乎不

受影响，错标只改变它周围的局部区域。因此，当怀疑训练数据被系统性地标错时，对于 SVM 来说，应该先清理数据或改用局部模型，调正则项不是解决问题的根本方法。

第四，SVM 和 KNN 的计算成本结构相反，需要根据实际部署方式选择。KNN 训练零成本，预测要遍历整份训练集且把它们全部存在内存里。SVM 训练则需要一定开销，但预测只需计算一次内积，且模型只需要 w 和 b 两个权重。对于在线低延迟、内存受限和数据持续增长的任务，可以选 SVM。一次性分析、训练集频繁更换、需要随时把新样本免费并入模型时 KNN 更佳。

最后，从可解释性上也可以对二者进行区分。SVM 的解释是全局性的，w 的分量直接排序特征重要性，且解释可以压缩到少量支持向量上。KNN 的解释则是局部的，以垃圾网页为例，某个页面被判为垃圾，是因为它的近邻大多是垃圾页。因此，个案解释 KNN 更直观，而全局机制解释 SVM 更完整。

附录 A　实现正确性对照结果

KNN 对照 KNeighborsClassifier，超参数直接取我们在验证集上选出的值，加权方式相同。SVM 对照线性核的 SVC，两者目标相同，均为 hinge loss 加上 L2 正则项。对照检验覆盖了合成数据实验的全部设置以及两个真实数据集上选定的最终超参数，双方使用完全相同的数据、切分与标准化。KNN 在 18 组设置上的预测与 sklearn 完全一致，准确率与 F1 的差异全部为 0。SVM 有 6 组指标完全一致，另有 8 组准确率差异不超过 0.5%。剩余 4 组差异较大的原因可以分为两类，一类是惩罚项系数 C 较小时的近似平局，双方训练目标值相差不到 1%，测试指标对边界位置敏感。另一类是高维数据下全批量梯度下降结果未收敛，指标相差 5 到 9 个百分点。

表 **A1**　实现正确性的合成实验对照结果

实验	设置	模型	我方 准确率/F1	sklearn 准确率/F1	差异 准确率/F1	一致
实验 1	$\sigma = 0.5$	KNN	1.0000 / 1.0000	1.0000 / 1.0000	0.0000 / 0.0000	是
	$\sigma = 0.5$	SVM	1.0000 / 1.0000	0.6333 / 0.4699	0.3667 / 0.5301	否
	$\sigma = 1$	KNN	0.9750 / 0.9764	0.9750 / 0.9764	0.0000 / 0.0000	是
	$\sigma = 1$	SVM	0.9708 / 0.9723	0.8917 / 0.8860	0.0792 / 0.0864	否
	$\sigma = 1.5$	KNN	0.9292 / 0.9323	0.9292 / 0.9323	0.0000 / 0.0000	是
	$\sigma = 1.5$	SVM	0.9167 / 0.9206	0.9167 / 0.9206	0.0000 / 0.0000	是
实验 2	单位圆与外环	KNN	1.0000 / 1.0000	1.0000 / 1.0000	0.0000 / 0.0000	是
	单位圆与外环	SVM	0.6292 / 0.6920	0.6250 / 0.6897	0.0042 / 0.0024	是
实验 3	$d = 3$	KNN	0.9850 / 0.9870	0.9850 / 0.9870	0.0000 / 0.0000	是
	$d = 3$	SVM	0.9950 / 0.9957	0.9950 / 0.9957	0.0000 / 0.0000	是
	$d = 10$	KNN	0.6850 / 0.7014	0.6850 / 0.7014	0.0000 / 0.0000	是
	$d = 10$	SVM	1.0000 / 1.0000	1.0000 / 1.0000	0.0000 / 0.0000	是
	$d = 50$	KNN	0.5950 / 0.6368	0.5950 / 0.6368	0.0000 / 0.0000	是
	$d = 50$	SVM	0.9950 / 0.9957	0.9950 / 0.9957	0.0000 / 0.0000	是
	$d = 100$	KNN	0.5550 / 0.6009	0.5550 / 0.6009	0.0000 / 0.0000	是
	$d = 100$	SVM	0.9900 / 0.9914	0.9850 / 0.9870	0.0050 / 0.0044	是
	$d = 200$	KNN	0.5000 / 0.5000	0.5000 / 0.5000	0.0000 / 0.0000	是
	$d = 200$	SVM	0.8400 / 0.8609	0.9300 / 0.9407	0.0900 / 0.0798	否
	$d = 500$	KNN	0.5050 / 0.4469	0.5050 / 0.4469	0.0000 / 0.0000	是
	$d = 500$	SVM	0.5300 / 0.5688	0.5800 / 0.6038	0.0500 / 0.0350	否
	$d = 1000$	KNN	0.5250 / 0.5852	0.5250 / 0.5852	0.0000 / 0.0000	是
	$d = 1000$	SVM	0.5300 / 0.5524	0.5300 / 0.5481	0.0000 / 0.0043	是
实验 4	$\eta = 0$	KNN	0.9800 / 0.9802	0.9800 / 0.9802	0.0000 / 0.0000	是
	$\eta = 0$	SVM	0.9940 / 0.9940	0.9940 / 0.9940	0.0000 / 0.0000	是

实验	设置	模型	我方 准确率/F1	sklearn 准确率/F1	差异 准确率/F1	一致
	$\eta = 0.02$	KNN	0.9760 / 0.9761	0.9760 / 0.9761	0.0000 / 0.0000	是
	$\eta = 0.02$	SVM	0.9640 / 0.9646	0.9640 / 0.9646	0.0000 / 0.0000	是
	$\eta = 0.05$	KNN	0.9700 / 0.9701	0.9700 / 0.9701	0.0000 / 0.0000	是
	$\eta = 0.05$	SVM	0.9280 / 0.9283	0.9300 / 0.9304	0.0020 / 0.0021	是
	$\eta = 0.08$	KNN	0.9660 / 0.9661	0.9660 / 0.9661	0.0000 / 0.0000	是
	$\eta = 0.08$	SVM	0.8820 / 0.8832	0.8840 / 0.8854	0.0020 / 0.0022	是
	$\eta = 0.12$	KNN	0.9640 / 0.9641	0.9640 / 0.9641	0.0000 / 0.0000	是
	$\eta = 0.12$	SVM	0.8420 / 0.8436	0.8420 / 0.8442	0.0000 / 0.0006	是

表 **A2** 实现正确性的真实实验对照结果

实验	设置	模型	我方 准确率/F1	sklearn 准确率/F1	差异 准确率/F1	一致
实验 7	webspam	KNN	0.9645 / 0.9711	0.9645 / 0.9711	0.0000 / 0.0000	是
实验 7	webspam	SVM	0.9307 / 0.9429	0.9305 / 0.9427	0.0002 / 0.0002	是
实验 7	HEPMASS	KNN	0.9036 / 0.9052	0.9036 / 0.9052	0.0000 / 0.0000	是
实验 7	HEPMASS	SVM	0.9071 / 0.9084	0.9069 / 0.9082	0.0002 / 0.0002	是

附录 **B** 复现说明

环境为 Python 3.11，依赖仅 NumPy 与 Matplotlib。各实验的运行命令如下。

```
python experiments/verify_sklearn.py                # 与 sklearn 对照
python experiments/exp{1_linear,2_circle,3_highdim}.py
python experiments/exp{4_label_noise,5_knn_detail,6_svm_detail}.py
python experiments/exp7_real.py [webspam|hepmass]  # 默认跑两个数据集
python experiments/exp7_replot.py                   # 按最新 CSV 绘图
```

随机种子全部固定为 42，涵盖合成数据生成、标签翻转、训练验证测试切分与真实数据分层抽样。实验 4 错标点的位置参数固定在脚本里。指标与图片直接写入 `results/`，本报告的图片引用 `results/figures/`，数字取自 `results/csv/`。